

Cahier des charges — Plateforme Universitaire Microservices (REST & SOAP)

Date : 27/11/2025

1. Contexte et Problématique

Contexte

Les universités gèrent plusieurs processus essentiels (inscription, suivi pédagogique, notes, paiements). Souvent, ces processus sont répartis sur des applications hétérogènes, difficiles à maintenir et à faire évoluer.

Problématique

Comment concevoir une plateforme unifiée et évolutive permettant l'authentification sécurisée des utilisateurs, la gestion des étudiants, la gestion des cours et emplois du temps via SOAP, la gestion des notes avec calcul de moyennes, la gestion de la facturation via SOAP, tout en assurant l'interopérabilité entre services, la sécurité (JWT) et un déploiement simple via Docker ?

2. Objectifs du Projet

Objectif général

Mettre en place une plateforme universitaire basée sur une architecture microservices mixte REST/SOAP, sécurisée par JWT et exposée via une API Gateway.

Objectifs spécifiques

- Mettre en place un service d'authentification (REST Spring Boot) générant des JWT et gérant les rôles (Admin, Enseignant, Étudiant).
- Développer un service REST Node.js/Express pour la gestion CRUD des étudiants.
- Développer un service SOAP Java JAX-WS pour la gestion des cours et emplois du temps (exposition d'un WSDL).
- Développer un service REST FastAPI pour la gestion des notes et le calcul automatique des moyennes.
- Développer un service SOAP .NET pour la facturation scolaire (frais, paiements, relevés).
- Centraliser l'accès via une API Gateway (routage, filtrage JWT, agrégation).
- Conteneuriser et déployer l'ensemble avec Docker Compose.

3. Périmètre (Scope)

Inclus

- Tous les microservices listés (Auth, Student, Course, Grade, Billing, Gateway).
- Communication interne via réseau Docker.

- JWT obligatoire sur les services REST avec contrôle par la Gateway.

Exclus (V1 étudiante)

- Interface front complète si non demandée.
- Orchestration avancée de type Kubernetes.
- Paiements bancaires réels (simulation acceptée).

4. Besoins Fonctionnels (BF)

BF1 — Auth■Service (REST)

- Enregistrer un utilisateur (register).
- Authentifier un utilisateur (login).
- Générer un token JWT signé.
- Vérifier la validité du JWT.
- Gérer les rôles : Admin / Enseignant / Étudiant.

BF2 — Student■Service (REST)

- Ajouter un étudiant.
- Modifier les informations d'un étudiant.
- Supprimer un étudiant.
- Lister tous les étudiants.
- Rechercher un étudiant par ID.
- Refuser toute requête sans JWT.

BF3 — Course■Service (SOAP)

- Ajouter / modifier / supprimer un cours.
- Lister les cours.
- Gérer les emplois du temps (horaires, salles).
- Exposer un WSDL.
- Répondre en XML SOAP.

BF4 — Grade■Service (REST)

- Ajouter / modifier / supprimer une note.
- Lister les notes d'un étudiant.
- Calculer automatiquement la moyenne.
- JWT obligatoire.

BF5 — Billing■Service (SOAP)

- Calculer les frais selon spécialité et niveau.
- Enregistrer / consulter les paiements.
- Générer un relevé de paiement.
- Indiquer si l'étudiant est en règle.
- Exposer WSDL et réponses XML SOAP.

BF6 — API Gateway

- Recevoir toutes les requêtes via un point unique.
- Vérifier le JWT avant routage.
- Router vers les services selon les chemins (/auth, /students, /courses, /grades, /billing).
- Masquer les adresses internes.
- Agréger plusieurs réponses si nécessaire.

BF7 — Déploiement Docker

- Un conteneur par service.
- Réseau interne pour communication inter■services.
- Ports séparés par service.
- Gateway exposée publiquement.
- Lancement de l'ensemble via docker■compose.

5. Besoins Non Fonctionnels (BNF)

Sécurité

- JWT obligatoire pour services sensibles.
- Chiffrement des mots de passe (BCrypt côté Auth).
- Gestion des rôles au niveau Auth et/ou Gateway.

Performance

- Réponses REST en JSON rapides.
- Latence acceptable < 1s pour opérations CRUD courantes.

Disponibilité & Fiabilité

- Chaque service doit pouvoir redémarrer indépendamment.
- Une panne d'un service ne doit pas bloquer les autres (sauf Auth).

Maintenabilité

- Code séparé par service et par responsabilité.
- Documentation et tests unitaires par service.

Interopérabilité

- REST ↔ JSON et SOAP ↔ XML/WSDL.
- Adaptateur SOAP→REST possible via Gateway si nécessaire.

Déployabilité

- Build et exécution via Docker, orchestration Compose.

6. Acteurs et Cas d'Utilisation (résumé)

Acteurs

- Admin : gère comptes, étudiants, cours.
- Enseignant : consulte étudiants, gère cours/notes.
- Étudiant : consulte ses cours, notes, statut de facturation.

7. Modèle de données & Diagramme de classes (UML)

Classes principales

- User(id, email, passwordHash, role)
- Student(id, matricule, nom, prenom, email, niveau, specialite)
- Course(id, code, titre, credit, enseignant, schedule)
- Schedule(id, jour, heureDebut, heureFin, salle)
- Grade(id, studentId, courseId, valeur, semestre)
- BillingRecord(id, studentId, montantTotal, montantPaye, statut)
- Payment(id, billingId, date, montant, mode)

Relations UML

- User (1) — (0..1) Student
- Student (1) — (0..*) Grade
- Course (1) — (0..*) Grade
- Course (1) — (0..1) Schedule
- Student (1) — (0..1) BillingRecord
- BillingRecord (1) — (0..*) Payment

Remarque d'implémentation

Dans l'architecture microservices, les services échangent surtout des identifiants (studentId, courseId) pour éviter des jointures entre bases de données.

8. Contraintes Techniques

- REST : Spring Boot (Auth, Gateway), Node.js/Express (Students), FastAPI (Grades).
- SOAP : Java JAX■WS (Courses), .NET Core C# (Billing).
- Docker : un conteneur par service + docker■compose.

9. Livrables attendus

- Code source de chaque service.
- docker-compose.yml complet.
- WSDL pour services SOAP.
- Collection Postman pour REST.
- Rapport final + manuel d'utilisation.